Vulnerability Report

Bonus — Sensitive Billing Data in CloudWatch Logs

DVSA-ORDER-BILLING Lambda — ICS 344 Course Project

- **Course / Section:** ICS 344 — Section _____
- **Student Name(s) + ID(s):** Faisal Alharbi — _____
- **Your DVSA Website URL:** http://dvsa-faisalalharbi-ics344-project-716522590330-us-east-1.s3-website.us-east-1.amazonaws.com
- **AWS Region:** us-east-1 (N. Virginia)
- **Date:** _____
- **Vulnerability Title(s):** Sensitive Billing Data Exposure in CloudWatch Logs (Bonus)

Bonus Vulnerability: Sensitive Billing Data in CloudWatch Logs

Part 1) Goal and Vulnerability Summary

This bonus vulnerability demonstrates that sensitive billing data submitted during the DVSA checkout process was written to Amazon CloudWatch Logs by the `DVSA-ORDER-BILLING` Lambda function. The affected components are AWS Lambda and CloudWatch Logs. The exposed data included payment-related fields such as card number (`ccn`), expiration date (`exp`), and CVV (`cvv`). The security impact is sensitive billing data exposure to any IAM user, role, or compromised identity with permission to read CloudWatch Logs.

Part 2) Why This Works / Root Cause

The vulnerability exists because the Lambda function logged the full incoming event object using:

```
print(json.dumps(event))
```

The event object contained the user's billing data:

```
"billing": {
    "ccn": "...",
    "exp": "...",
    "cvv": "..."
}
```

Because Lambda stdout is stored in CloudWatch Logs, the full billing object became visible in the log stream. The root cause is unsafe debugging/logging of full request data without redacting sensitive fields.

Part 3) Environment and Setup

```
AWS Region:        us-east-1 / N. Virginia
DVSA Website:      dvsa-faisalalharbi-ics344-project-716522590330-us-east-1
                   .s3-website.us-east-1.amazonaws.com
Lambda Function:   DVSA-ORDER-BILLING
Source File:       order_billing.py
CloudWatch Group:  /aws/lambda/DVSA-ORDER-BILLING
Test User:         Normal non-admin DVSA user
Tools Used:        DVSA web UI, AWS Console, Lambda Console, CloudWatch Logs
```

Fake billing data was used for testing only.

Part 4) Reproduction Steps

1. Logged in to DVSA as a normal user.
2. Added a product to the cart.
3. Continued through the checkout flow.
4. Submitted shipping details.
5. Submitted fake billing details containing card number, expiration date, and CVV.
6. Opened AWS Console -> CloudWatch -> Logs -> Log groups.
7. Opened the log group `/aws/lambda/DVSA-ORDER-BILLING`.
8. Opened the newest log stream after the billing request.
9. Observed that the log stream contained the full billing object, including `ccn`, `exp`, and `cvv`.

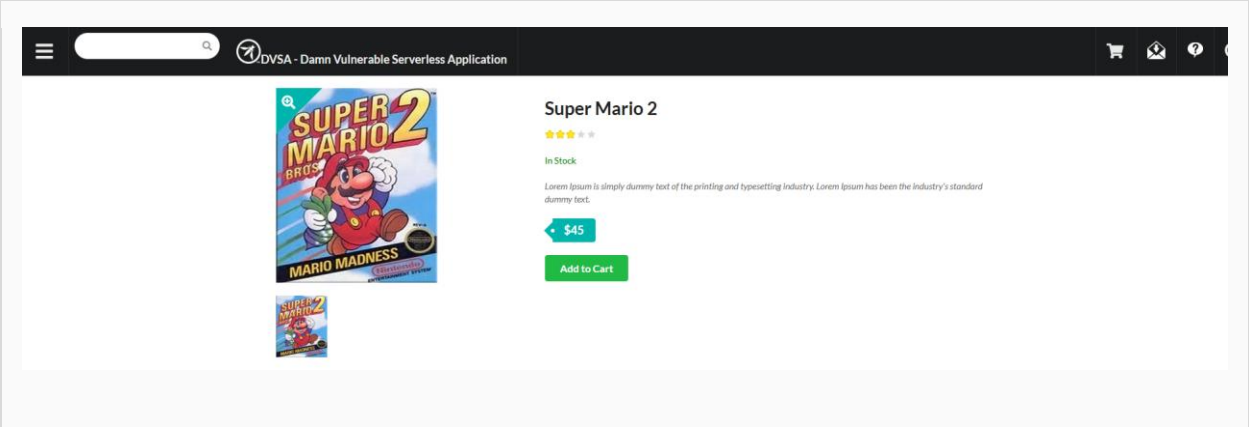

Figure 1: *Cart page during checkout flow.*

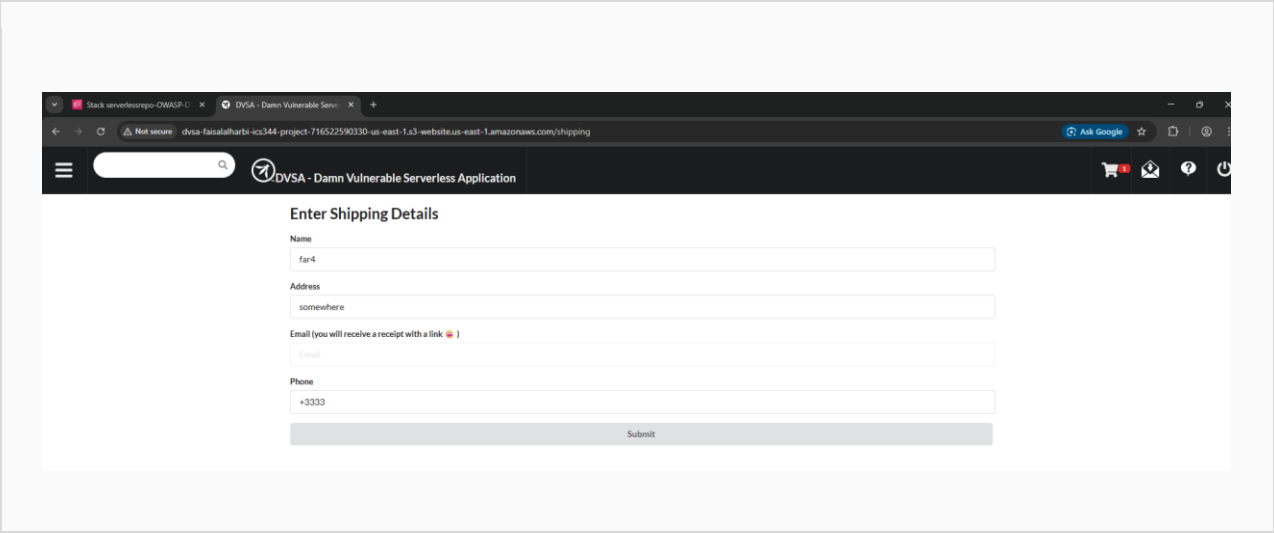

Figure 2: *Shipping details submission.*

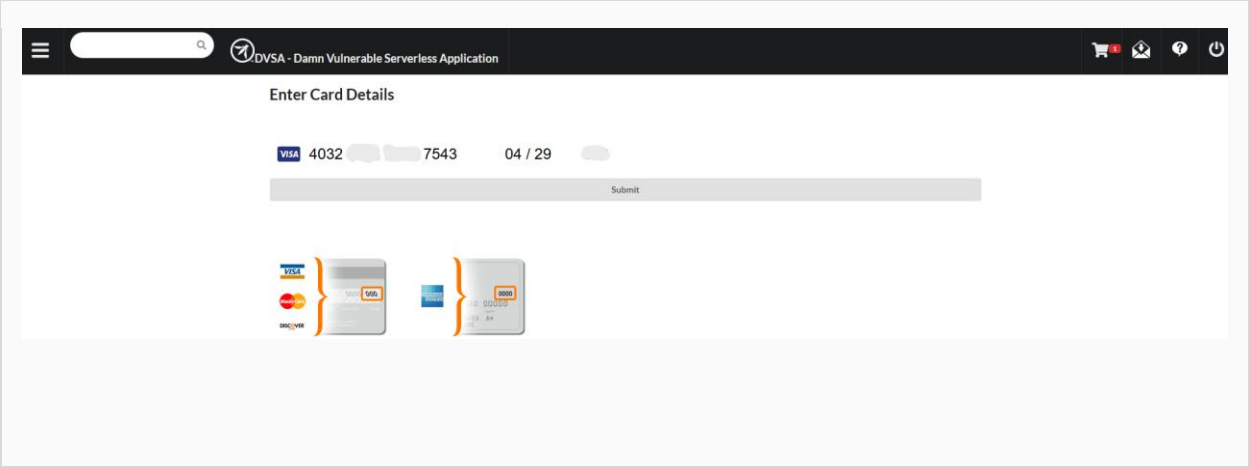

Figure 3: *Fake billing details submitted at the card entry step.*

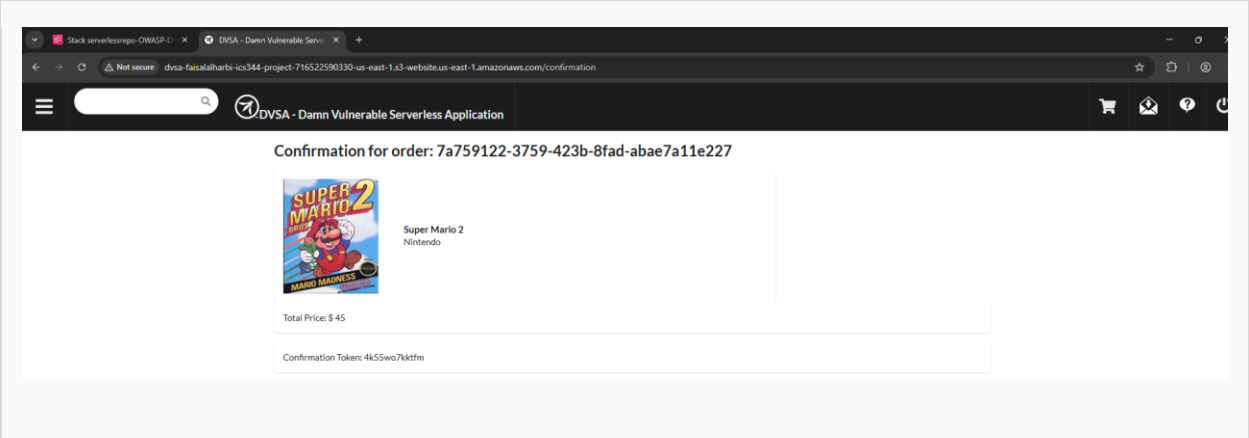

Figure 4: *Order confirmation page.*

Part 5) Evidence and Proof

Before the fix, CloudWatch Logs showed the billing object in plaintext:

```
"billing": {
    "ccn": "4032*******7543",
    "exp": "04/29",
    "cvv": "***"
}
```

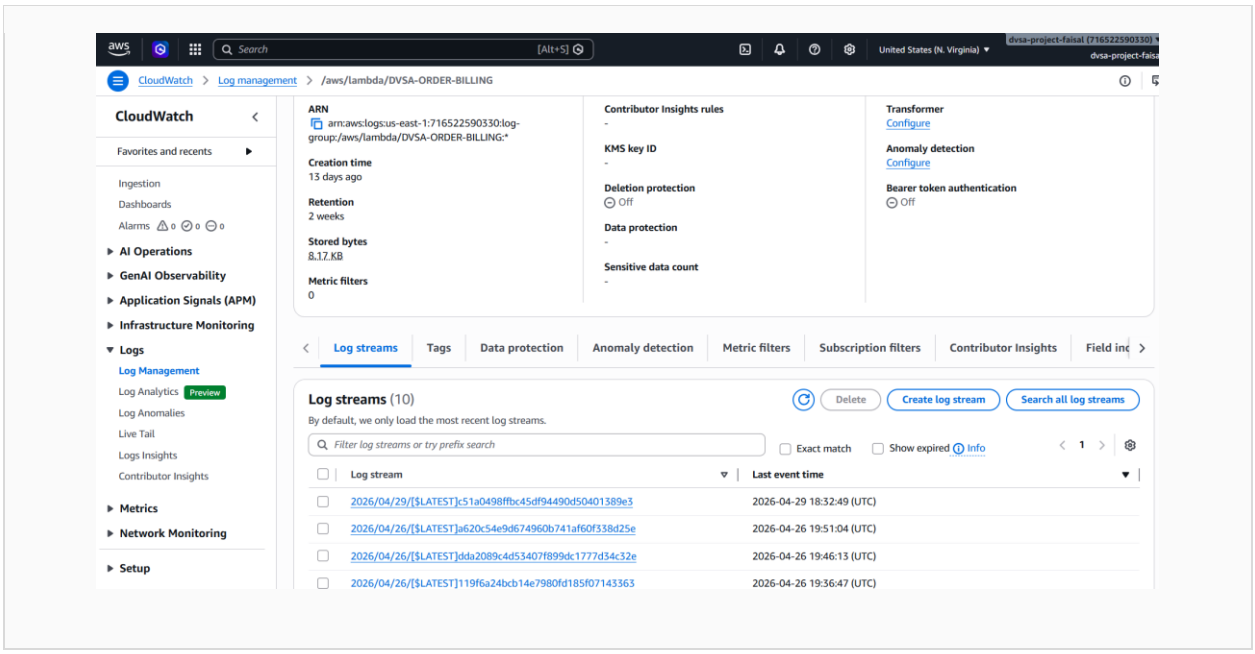

Figure 5: *CloudWatch log group /aws/lambda/DVSA-ORDER-BILLING.*

Figure 6: *Log stream containing the full billing object with ccn, exp, and cvv before the fix.*

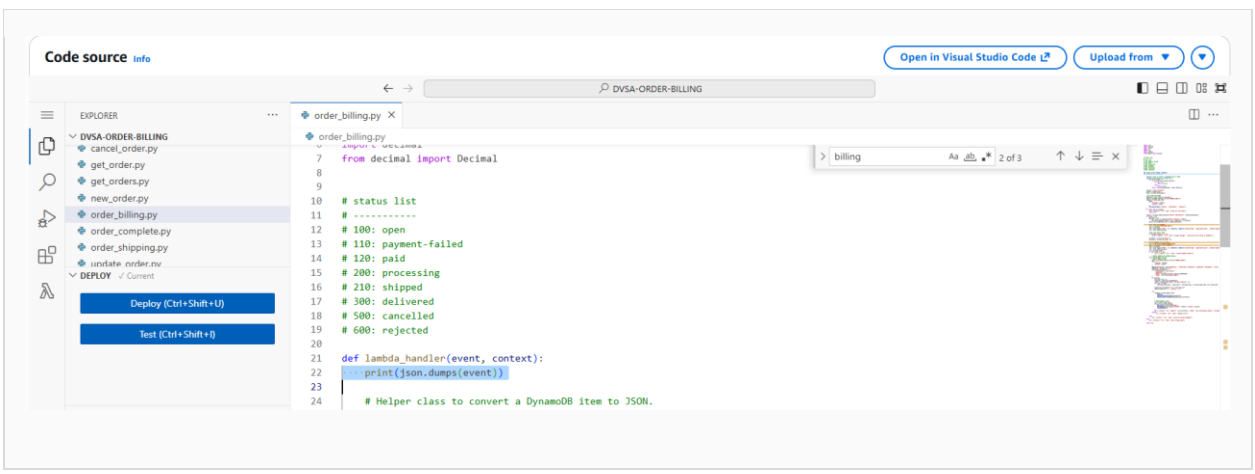

Figure 7: *Vulnerable source line in order_billing.py: print(json.dumps(event)).*

The screenshots prove that billing data appeared in CloudWatch Logs and show the vulnerable source code line:

```
print(json.dumps(event))
```

Part 6) Fix Strategy / Probable Mitigation

The fix should be applied in the Lambda logging logic. The function should not log full request objects or raw billing data. Instead, sensitive fields should be redacted before logging.

Sensitive fields include:

```
ccn
cvv
cvc
exp
expiry
expiration
card
cardnumber
creditcard
authorization
token
password
```

CloudWatch Logs access should also be restricted using least privilege, and log retention should be configured.

Part 7) Code / Config Changes

The vulnerable line was removed:

```
print(json.dumps(event))
```

It was replaced with a recursive redaction helper:

```
def redact_sensitive_fields(value):
    sensitive_keys = {
        "ccn",
        "cvv",
        "cvc",
        "exp",
        "expiry",
        "expiration",
        "card",
        "cardnumber",
```

```
        "creditcard",
        "authorization",
        "token",
        "password"
    }

    if isinstance(value, dict):
        redacted = {}
        for key, val in value.items():
            if str(key).lower() in sensitive_keys:
                redacted[key] = "[REDACTED]"
            else:
                redacted[key] = redact_sensitive_fields(val)
        return redacted

    if isinstance(value, list):
        return [redact_sensitive_fields(item) for item in value]

    return value

print(json.dumps(redact_sensitive_fields(event)))
```

```python
def lambda_handler(event, context):
    #===START===
    # SECURITY FIX:
    # Do not log the full event directly because it contains sensitive billing data.
    # Redact card number, CVV, expiry date, tokens, and passwords before logging.
    def redact_sensitive_fields(value):
        sensitive_keys = {
            "ccn",
            "cvv",
            "cvc",
            "exp",
            "expiry",
            "expiration",
            "card",
            "cardnumber",
            "creditcard",
            "authorization",
            "token",
            "password"
        }

        if isinstance(value, dict):
            redacted = {}
            for key, val in value.items():
                if str(key).lower() in sensitive_keys:
                    redacted[key] = "[REDACTED]"
                else:
                    redacted[key] = redact_sensitive_fields(val)
            return redacted
```

```python
        if isinstance(value, list):
            return [redact_sensitive_fields(item) for item in value]

        return value

    print(json.dumps(redact_sensitive_fields(event)))

    #===END===
    # Helper class to convert a DynamoDB item to JSON.
    class DecimalEncoder(json.JSONEncoder):
```

Figure 8: *Updated order_billing.py showing the redaction helper applied before logging.*

Part 8) Verification After Fix

After deploying the fixed Lambda code, a new billing request was submitted using new fake test data. The newest CloudWatch log stream was checked again.

After the fix, the log output showed:

```
"billing": {
    "ccn": "[REDACTED]",
    "exp": "[REDACTED]",
    "cvv": "[REDACTED]"
}
```

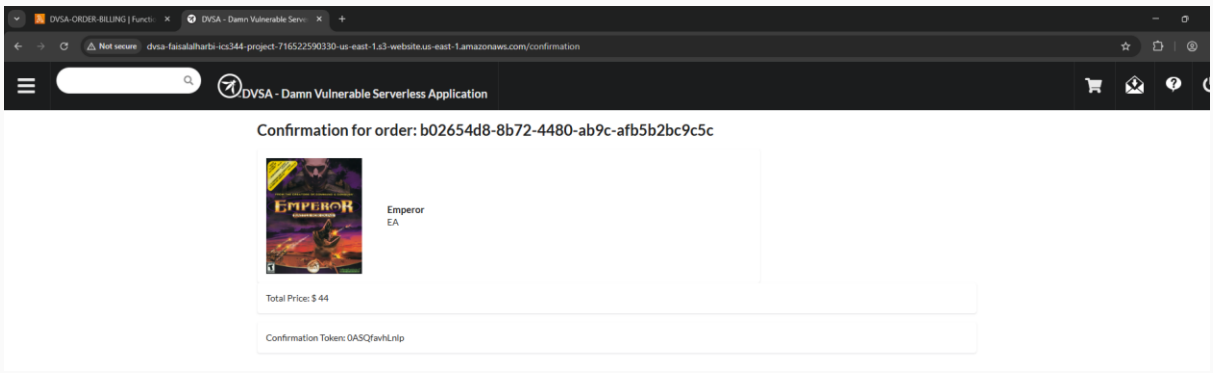

Figure 9: *New billing request submitted after deploying the fix.*

Figure 10: *CloudWatch log stream after the fix showing redacted billing fields.*

This confirms that sensitive billing fields are no longer exposed in CloudWatch Logs, while the billing Lambda still runs normally.

Part 9) Structured Operation and Security Analysis

Table A - Intended Logic and Behavior Trace

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence

Sensitive billing data in CloudWatch logs	Billing data may be processed by the backend, but card number, expiry, and CVV must not be written to persistent logs. Logs should contain only safe operational data.	DVSA checkout flow, order_billing.py, CloudWatch log stream, Lambda source code, billing request screenshots	User submits billing details and the order billing function processes the request.	CloudWatch log stream showed the full billing object containing ccn, exp, and cvv.

Table B - Deviation and Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied Where	Post-Fix Verification	Optional Latency
Sensitive billing data in CloudWatch logs	The backend wrote sensitive payment fields to CloudWatch Logs. This violates secure logging and data minimization because users with log-read permissions could view billing data.	Accidental misconfiguration / security-relevant abuse	DVSA-ORDER-BILLING, file order_billing.py; replaced print(json.dumps(event)) with redacted logging.	New billing request produced CloudWatch logs where ccn, exp, and cvv appeared as [REDACTED].	Optional

Part 10) Takeaway / Lessons Learned

CloudWatch Logs are part of the security boundary in serverless applications. Logging full Lambda events may expose sensitive user input, tokens, payment data, or internal identifiers. Secure serverless applications should use structured logging, redact sensitive fields, restrict log access, and avoid storing unnecessary sensitive data in logs.

Note:

This bonus finding is different from official Lesson #3. Lesson #3 focuses on sensitive information disclosure through other channels; this finding specifically targets sensitive payment data leakage through Lambda stdout written to CloudWatch Logs by the DVSA-ORDER-BILLING function.